How-to named pipes

What Are Named Pipes (FIFO)?

A **named pipe**, or **FIFO (First-In-First-Out)**, is a special type of file used for **Inter-Process Communication (IPC)**. It allows processes to communicate by writing and reading data, just like an anonymous pipe (|), but with some key differences:

1. **Named pipes exist as actual files** in the filesystem and can be accessed by multiple unrelated processes.
2. **They persist beyond process execution** until deleted manually.
3. **They work in a first-in, first-out manner**, meaning data is read in the order it was written.
4. **A named pipe is unidirectional** (one-way communication). If you need bidirectional communication, you need two FIFOs.

1. Creating a Named Pipe (FIFO)

Named pipes can be created in two ways:

Method 1: Using the Terminal

You can use the `mkfifo` command:

```
mkfifo myfifo
```

This creates a special file called `myfifo`. You can verify it using:

```
ls -l
```

You'll see something like:

```
prw-r--r-- 1 user user 0 Feb 27 12:00 myfifo
```

`p` at the beginning indicates it is a **pipe**.

Method 2: Using C Code

The `mkfifo()` system call is used to create a FIFO file programmatically.

```
#include <sys/types.h>
#include <sys/stat.h>
#include <stdio.h>

#define FIFO_PATH "myfifo"

int main() {
    // Create named pipe with read and write permissions
    if (mkfifo(FIFO_PATH, 0666) == -1) {
        perror("mkfifo failed");
        return 1;
    }
    printf("FIFO created successfully.\n");
    return 0;
}
```

Compile and run:

```
gcc create_fifo.c -o create_fifo
./create_fifo
```

2. Writing to a Named Pipe (Writer Process)

The writer process will open the FIFO for writing and send a message.

C Code for Writer

```
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <sys/stat.h>
#include <sys/types.h>
#include <unistd.h>

#define FIFO_PATH "myfifo"

int main() {
    int fd;
    char buffer[100];

    printf("Writer: Opening FIFO for writing...\n");
    fd = open(FIFO_PATH, O_WRONLY);  // Open FIFO for writing
```

```
    printf("Writer: Enter a message: ");
    fgets(buffer, sizeof(buffer), stdin);

    write(fd, buffer, sizeof(buffer));  // Write to FIFO
    printf("Writer: Message sent.\n");

    close(fd);  // Close FIFO
    return 0;
}
```

3. Reading from a Named Pipe (Reader Process)

The reader process will open the FIFO for reading and print the received message.

C Code for Reader

```
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <sys/stat.h>
#include <sys/types.h>
#include <unistd.h>

#define FIFO_PATH "myfifo"

int main() {
    int fd;
    char buffer[100];

    printf("Reader: Opening FIFO for reading...\n");
    fd = open(FIFO_PATH, O_RDONLY);  // Open FIFO for reading

    read(fd, buffer, sizeof(buffer));  // Read from FIFO
    printf("Reader: Received message: %s", buffer);

    close(fd);  // Close FIFO
    return 0;
}
```

4. Running the Programs

Step 1: Compile Both Programs

```
gcc writer.c -o writer
gcc reader.c -o reader
```

Step 2: Run the Reader in One Terminal

```
./reader
```

It will block and wait for input because there's no writer yet.

Step 3: Run the Writer in Another Terminal

```
./writer
```

Enter a message, e.g., `Hello FIFO!`, and press enter.

Step 4: Observe the Output in the Reader Terminal

```
Reader: Received message: Hello FIFO!
```

5. Important Notes

1. **FIFO is blocking by default**
   - The reader process waits for the writer.
   - The writer process waits for the reader.
2. **Multiple Writers & Single Reader**
   - If multiple processes write to the same FIFO, data may interleave.
3. **Non-Blocking Mode**
   - You can use `O_NONBLOCK` flag to prevent blocking:

     fd = open(FIFO_PATH, O_RDONLY | O_NONBLOCK);

4. **Deleting a Named Pipe**
   - Remove it when done:

     rm myfifo

`dup()` and `dup2()` in Linux

In Linux, `dup()` and `dup2()` are system calls that allow us to duplicate file descriptors, which is useful for **redirecting standard input/output (stdin, stdout, stderr), inter-process communication (IPC), and managing multiple file streams.**

1. `dup()` - Duplicate a File Descriptor

Syntax

```
#include <unistd.h>

int dup(int oldfd);
```

- `oldfd`: The existing file descriptor to be duplicated.
- **Returns**: A new file descriptor (lowest available) that refers to the same file.
- Both file descriptors share the **same file offset and status flags**.

Example: Redirect `stdout` to a File

```
#include <stdio.h>
#include <unistd.h>
#include <fcntl.h>

int main() {
    int fd = open("output.txt", O_WRONLY | O_CREAT | O_TRUNC,
0644);
    if (fd == -1) {
        perror("open failed");
        return 1;
    }

    int copy_fd = dup(fd);  // Duplicates `fd`
    write(copy_fd, "Hello via dup()!\n", 17);

    close(fd);
    close(copy_fd);
    return 0;
}
```

Key Takeaways:

- `dup(fd)` returns a new file descriptor that behaves like `fd`.
- Writing to `copy_fd` writes to `output.txt`.

2. `dup2()` - Duplicate a File Descriptor to a Specific FD

Syntax

```
#include <unistd.h>

int dup2(int oldfd, int newfd);
```

- `oldfd`: The file descriptor to duplicate.
- `newfd`: The desired file descriptor number.
- If `newfd` is **already open**, it is **closed first** before duplication.

Example: Redirect `stdout` to a File

```
#include <stdio.h>
#include <unistd.h>
#include <fcntl.h>

int main() {
    int fd = open("output.txt", O_WRONLY | O_CREAT | O_TRUNC,
0644);
    if (fd == -1) {
        perror("open failed");
        return 1;
    }

    dup2(fd, STDOUT_FILENO);  // Redirect stdout to file
    printf("This will be written to output.txt instead of the
terminal\n");

    close(fd);
    return 0;
}
```

Key Takeaways:

- `dup2(fd, STDOUT_FILENO)` redirects `stdout` (1) to `fd`, so `printf()` writes to the file.
- The original `stdout` is lost, so restore it if needed.

3. `dup()` and `dup2()` with Named Pipes (FIFO)

Now, let's use **named pipes** with `dup2()` for IPC between two processes.

Example: Process A Writes to FIFO, Process B Reads Using `dup2()`

Step 1: Create a Named Pipe

```
mkfifo myfifo
```

Step 2: Writer Program (Process A)

```
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>

int main() {
    int fd = open("myfifo", O_WRONLY);
    if (fd == -1) {
        perror("open failed");
        return 1;
    }

    dup2(fd, STDOUT_FILENO);  // Redirect stdout to FIFO
    printf("Hello from Process A!\n");

    close(fd);
    return 0;
}
```

What happens?

- `stdout` (1) is redirected to `myfifo`, so `printf()` writes to the named pipe.

Step 3: Reader Program (Process B)

```
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>

int main() {
    int fd = open("myfifo", O_RDONLY);
    if (fd == -1) {
        perror("open failed");
```

```
        return 1;
    }

    dup2(fd, STDIN_FILENO);  // Redirect stdin to FIFO
    char buffer[100];
    fgets(buffer, sizeof(buffer), stdin);  // Read from FIFO
    printf("Received: %s", buffer);

    close(fd);
    return 0;
}
```

Running the Example

1. **Compile both programs**

```
gcc writer.c -o writer
gcc reader.c -o reader
```

2. **Run the reader first** (it waits for input):bash

```
./reader
```

3. **Run the writer**:bash

```
./writer
```

4. The reader should print:

```
Received: Hello from Process A!
```

Summary

Function	Description
`dup(fd)`	Duplicates `fd` to the lowest available descriptor.
`dup2(oldfd, newfd)`	Duplicates `oldfd` into `newfd`, closing `newfd` first if needed.
`dup2()` with FIFO	Redirects `stdout` or `stdin` to a named pipe for IPC.